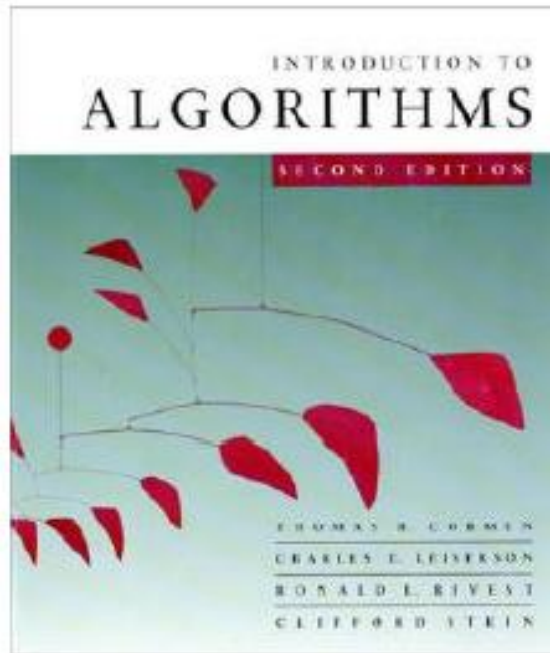


Introduction to Algorithms



Chapter 1: The Role of Algorithms in Computing

Computational problems

- A computational problem specifies an input-output relationship
 - What does the input look like?
 - What should the output be for each input?
- Example:
 - Input: an integer number n
 - Output: Is the number prime?
- Example:
 - Input: A list of names of people
 - Output: The same list sorted alphabetically

Algorithms

- A tool for solving a well-specified computational problem



- Algorithms must be:
 - Correct: For each input produce an appropriate output
 - Efficient: run as quickly as possible, and use as little memory as possible – more about this later

Algorithms Cont.

- A well-defined **computational procedure** that takes some value, or set of values, as **input** and produces some value, or set of values, as **output**.
- Written in a **pseudo code** which can be implemented in the language of programmer's choice.

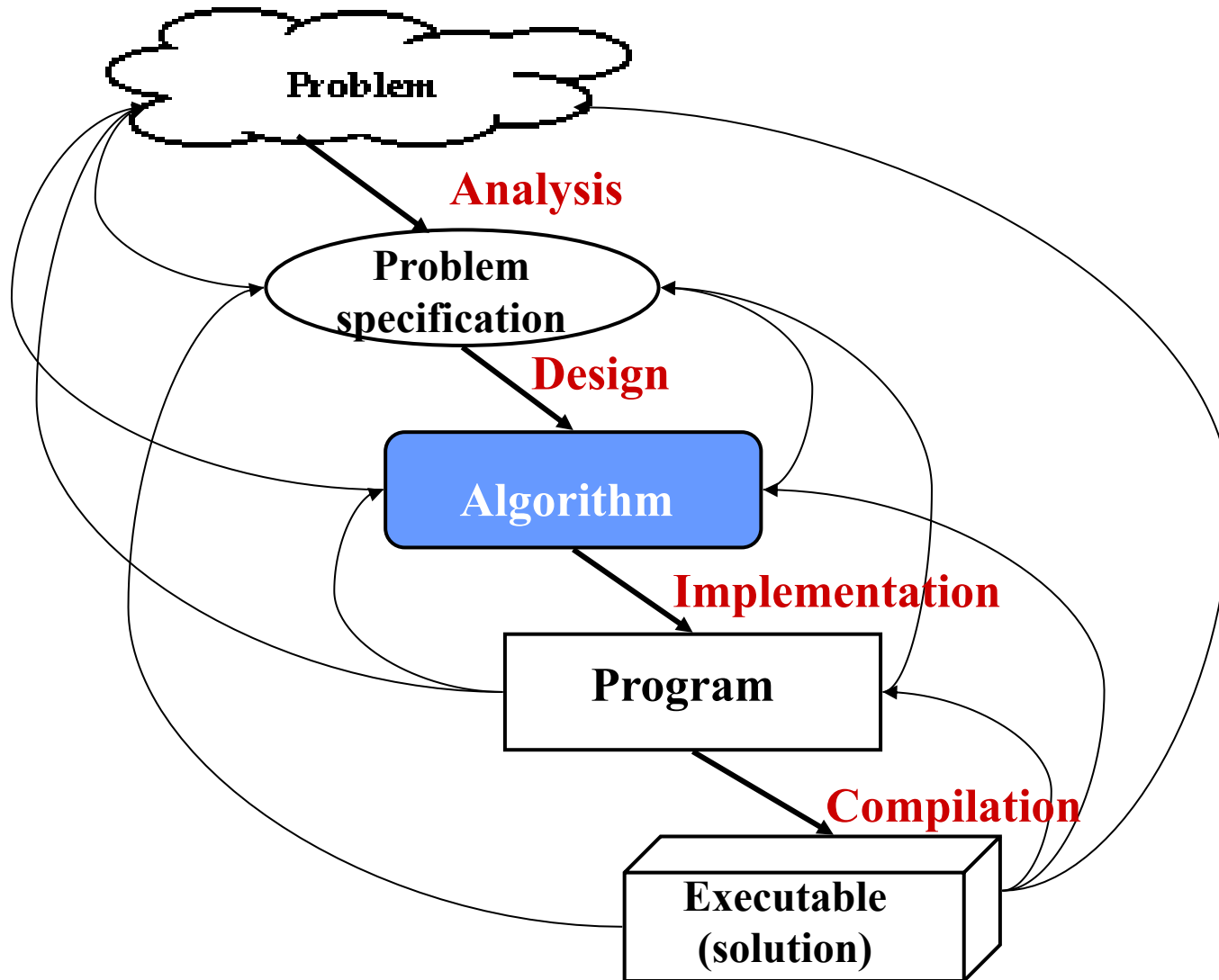
Correct and incorrect algorithms

- Algorithm is correct if, for every input instance, it ends with the correct output. We say that a correct algorithm solves the given computational problem.
- An incorrect algorithm **might not end** at all on some input instances, or it might end with an answer other than the desired one.
- We shall be concerned only with correct algorithms.

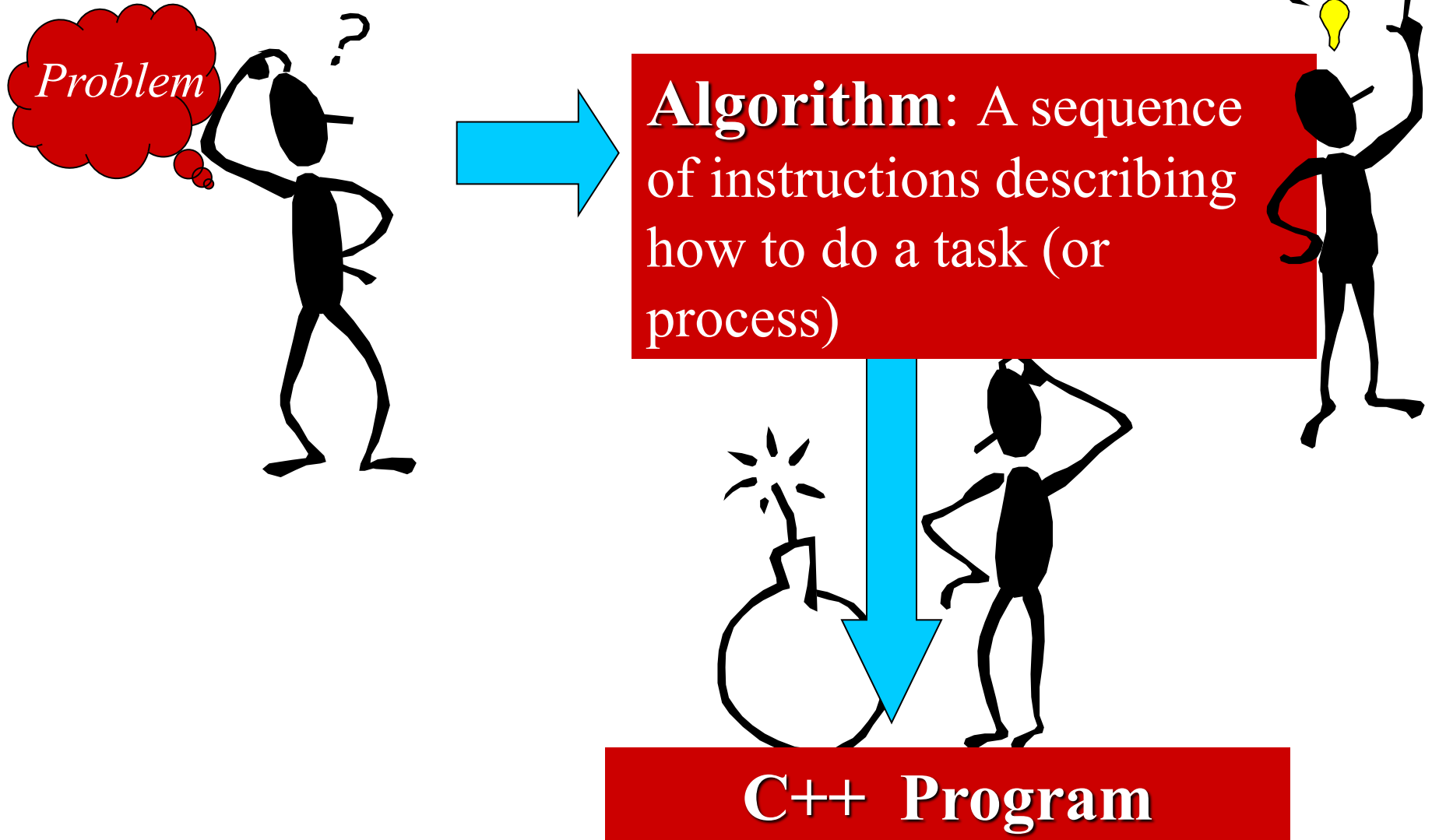
Problems and Algorithms

- We need to solve a computational problem
 - “Convert a weight in pounds to Kg”
- An algorithm specifies how to solve it, e.g.:
 - 1. Read weight-in-pounds
 - 2. Calculate $\text{weight-in-Kg} = \text{weight-in-pounds} * 0.455$
 - 3. Print weight-in-Kg
- A computer program is a computer-executable description of an algorithm

The Problem-solving Process



From Algorithms to Programs



Practical Examples

■ Internet and Networks

- The need to access large amount of information with the **shortest time**.
- Problems of finding the **best routs** for the data to travel.
- Algorithms for **searching** this large amount of data to quickly find the pages on which particular information resides.

■ Electronic Commerce

- The ability of **keeping the information** (credit card numbers, passwords, bank statements) private, safe, and secure.
- Algorithms involves **encryption/decryption** techniques.

Hard problems

- We can **identify the Efficiency** of an algorithm from its **speed** (how long does the algorithm take to **produce the result**).
- Some problems have **unknown efficient** solution.
- These problems are called NP-complete problems.
- If we can show that the problem is NP-complete, we can **spend our time** developing an **efficient algorithm** that gives a good, but not the best possible solution.

Components of an Algorithm

- Variables and values
- Instructions
- Sequences
 - A series of instructions
- Procedures
 - A named sequence of instructions
 - we also use the following words to refer to a “Procedure” :
 - Sub-routine
 - Module
 - Function

Components of an Algorithm Cont.

■ Selections

- ❑ An instruction that decides which of two possible sequences is executed
- ❑ The decision is based on true/false condition

■ Repetitions

- ❑ Also known as iteration or loop

■ Documentation

- ❑ Records what the algorithm does

A Simple Algorithm

- INPUT: a sequence of n numbers
 - T is an array of n elements
 - $T[1], T[2], \dots, T[n]$
- OUTPUT: the smallest number among them

```
min = T[0]
for i = 0 to n-1 do
{
    if T[i] < min
        min = T[i]
}
Output min
```

- **Performance** of this algorithm is a function of **n**

Example

- Code:
- `a = b;`
- Complexity: ?

Example

- Code:

- ```
sum = 0;
```
- ```
for (i=1; i <=n; i++)
```
- ```
 sum += n;
```

- Complexity:

# Example

- Code:

- `sum = 0;` - 1
- `for (i=1; i<=n; i++)` - n+1
- `sum += n;` - n

- Complexity:

$$\begin{aligned}T(n) &= 1 + n + 1 + n \\&= 2n + 2 \\&= O(n)\end{aligned}$$

# Example

- Code:
  - `sum1 = 0;`
  - `for (i=1; i<=n; i++)`
  - `for (j=1; j<=n; j++)`
  - `sum1++;`
- Complexity:

# Algorithm Efficiency

- Consider two sorting algorithms
  - Insertion sort
    - takes  $c_1 \cdot n^2$  to sort  $n$  items
    - where  $c_1$  is a constant that does not depend on  $n$
    - it takes time roughly proportional to  $n^2$
  - Merge Sort
    - takes  $c_2 \cdot n \lg(n)$  to sort  $n$  items
    - where  $c_2$  is also a constant that does not depend on  $n$
    - $\lg(n)$  stands for  $\log_2(n)$
    - it takes time roughly proportional to  $n \lg(n)$
- Insertion sort usually has a smaller constant factor than merge sort
  - so that,  $c_1 < c_2$
- Merge sort is faster than insertion sort for large input sizes

# Algorithm Efficiency Cont.

- Consider now:

- A **faster** computer **A** running **insertion sort** against
- A **slower** computer **B** running **merge sort**
- **Both must sort an array of one million ( $10^6$ ) numbers**

- Suppose

- Computer **A** execute **one billion** ( $10^9$ ) instructions per second
- Computer **B** execute **ten million** ( $10^7$ ) instructions per second
- So computer **A** is **100** times faster than computer **B**

- Assume that

- $c_1 = 2$                       and                       $c_2 = 50$

# Algorithm Efficiency Cont.

- To sort one million numbers

- Computer **A** takes

$$\frac{2 \cdot (10^6)^2 \text{ instructions}}{10^9 \text{ instructions/second}}$$

$$= 2000 \text{ seconds}$$

$$n = 10^6$$

insertion sort :  $c1 \cdot n^2$

- Computer **B** takes

$$\frac{50 \cdot 10^6 \cdot \lg(10^6) \text{ instructions}}{10^7 \text{ instructions/second}}$$

$$\approx 100 \text{ seconds}$$

Merge sort :  $c2 \cdot n \lg(n)$

- By using algorithm whose running time grows more slowly, Computer B runs 20 times faster than Computer A
- For ten million numbers
  - Insertion sort takes  $\approx 2.3$  days
  - Merge sort takes  $\approx 20$  minutes

# Pseudo-code conventions

Algorithms are typically written in **pseudo-code** that is similar to **C/C++** and **JAVA**.

- Pseudo-code differs from real code with:
  - It is not typically concerned with issues of **software engineering**.
  - Issues of **data abstraction**, and **error** handling are often **ignored**.
- **Indentation** indicates **block** structure.
- The **symbol** "**▷**" indicates that the remainder of the line is a **comment**. Comment is also denoted as **"//"**
- **A multiple assignment** of the form  $i \leftarrow j \leftarrow e$  assigns to both variables  $i$  and  $j$  the value of expression  $e$ ; it should be treated as equivalent to the assignment  $j \leftarrow e$  followed by the assignment  $i \leftarrow j$ .

# Pseudo-code conventions

- Variables ( such as  $i$ ,  $j$ , and  $key$ ) are **local** to the given procedure. We shall not use **global variables without** explicit indication.
- Array elements are accessed by specifying the array name followed by the index in square brackets. For example,  **$A[i]$**  indicates the  $i^{th}$  element of the array  $A$ . The notation “...” is used to indicate a range of values within an array. Thus,  **$A[1...j]$**  indicates the sub-array of  $A$  consisting of the  $j$  elements  $A[1]$ ,  $A[2]$ , . . . ,  $A[j]$ .
- A particular attributes is accessed using the attributes name followed by the name of its **object** in square brackets.
  - For example, we treat an array as an object with the attribute length indicating how many elements it contains(  **$length[A]$** ).

# Pseudo-code Example

```
MERGE(A, p, q, r)
1 $n_1 \leftarrow q - p + 1$
2 $n_2 \leftarrow r - q$
3 ▸ create arrays $L[1 \dots n_1 + 1]$ and $R[1 \dots n_2 + 1]$
4 for $i \leftarrow 1$ to n_1
5 do $L[i] \leftarrow A[p + i - 1]$
6 for $j \leftarrow 1$ to n_2
7 do $R[j] \leftarrow A[q + j]$
8 $L[n_1 + 1] \leftarrow \infty$
9 $R[n_2 + 1] \leftarrow \infty$
10 $i \leftarrow 1$
11 $j \leftarrow 1$
12 for $k \leftarrow p$ to r
13 do if $L[i] \leq R[j]$
14 then $A[k] \leftarrow L[i]$
15 $i \leftarrow i + 1$
16 else $A[k] \leftarrow R[j]$
17 $j \leftarrow j + 1$
```